LAPORAN PRAKTIKUM 2

SINKRONISASI DAN DEADLOCK MENGGUNAKAN OPENMP

PEMROSESAN PARALEL

Dosen Pengampu : Huda Ubaya, M.T.



Disusun Oleh :

Sachio Aji

NIM 09011482528013

DEPARTEMEN SISTEM KOMPUTER

FAKULTAS ILMU KOMPUTER

UNIVERSITAS SRIWIJAYA

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman paralel dengan shared memory, beberapa thread bisa membaca dan mengubah lokasi memori yang sama pada saat yang bersamaan. Kalau akses ini tidak dijaga dengan mekanisme yang tepat, terjadilah race condition: hasil akhir program menjadi salah dan bisa berbeda setiap kali program dijalankan. OpenMP punya beberapa cara untuk menghindarinya, yaitu direktif critical, direktif atomic, klausa reduction, dan teknik akumulasi ke array lokal milik tiap thread. Biaya masing-masing cara berbeda, karena seberapa sering thread saling berebut (kontensi) juga berbeda.

Masalah lain muncul dari penggunaan kunci (lock) yang kurang hati-hati, yaitu deadlock. Pada kondisi ini, sekelompok thread berhenti selamanya karena masing-masing menunggu kunci yang sedang dipegang thread lain. Praktikum ini memakai dua kasus uji. Kasus pertama adalah menghitung histogram 256 bin dari 10^8 byte acak, untuk melihat apakah hasilnya benar dan seberapa besar pengaruh tiap metode sinkronisasi terhadap kinerja. Kasus kedua adalah simulasi pemindahan dana antara dua rekening, untuk memancing deadlock lewat urutan penguncian yang berbeda, lalu mencegahnya dengan urutan penguncian yang seragam.

1.2 Tujuan Praktikum

1. Membuktikan adanya race condition pada pembaruan histogram dengan data hasil eksekusi berulang.
2. Memperbaiki race condition dengan critical, atomic, reduction, dan akumulasi lokal per thread.
3. Menjelaskan perbedaan kinerja keempat perbaikan dengan konsep rebutan (kontensi) dan biaya sinkronisasi.
4. Mereproduksi deadlock dengan dua lock dan mencegahnya dengan urutan penguncian yang tetap.

BAB II

DASAR TEORI

2.1 Race Condition

Race condition muncul saat dua thread atau lebih membaca dan menulis lokasi memori yang sama tanpa ada sinkronisasi. Satu baris h[k]++ ternyata dikerjakan prosesor dalam tiga langkah: membaca nilai h[k], menambahkannya satu, lalu menyimpannya kembali. Bayangkan dua thread sama-sama membaca nilai lama sebelum salah satunya sempat menyimpan. Keduanya akan menyimpan nilai lama ditambah satu, sehingga hasilnya hanya naik satu, padahal seharusnya naik dua. Satu penambahan pun hilang (lost update). Urutan penjadwalan thread tidak bisa diprediksi, jadi jumlah penambahan yang hilang selalu berbeda setiap kali program dijalankan.

2.2 Mekanisme Sinkronisasi pada OpenMP

- critical: hanya satu thread yang boleh masuk ke wilayah kritis pada satu waktu, sementara thread lain menunggu giliran. Cara ini paling gampang dipakai, tetapi paling mahal kalau diletakkan di dalam perulangan yang iterasinya sangat banyak.

- atomic: menjaga satu pembaruan sederhana pada satu lokasi memori dengan instruksi atomik dari perangkat keras, sehingga tidak perlu antre kunci.

- reduction: tiap thread mendapat salinan privat dari variabel (atau array), lalu semua salinan itu digabung di akhir wilayah paralel. Untuk reduction pada array, dibutuhkan OpenMP 4.5 ke atas.

- Akumulasi lokal per thread: tiap thread mengisi histogram lokal miliknya sendiri, lalu hasilnya digabung ke histogram utama satu kali saja di akhir**.** Selama perulangan utama berjalan, tidak ada rebutan memori bersama.

2.3 Kontensi dan Biaya Sinkronisasi

Kontensi adalah kondisi ketika beberapa thread berebut sumber daya yang sama, baik itu kunci maupun baris cache. Makin sering thread saling berebut, makin

banyak waktu yang habis untuk menunggu dan untuk memindahkan baris cache bolak-balik antar core (cache-line bouncing). Karena itu, metode yang sama sekali menghindari rebutan, yaitu reduction dan akumulasi lokal, adalah yang tercepat. Sebaliknya, critical yang memaksa thread antre di setiap iterasi adalah yang paling lambat.

2.4 Deadlock dan Syarat Circular Wait

Deadlock terjadi ketika dua thread atau lebih sama-sama tidak bisa lanjut, karena masing-masing memegang kunci yang dibutuhkan thread lain, sehingga mereka saling menunggu dalam lingkaran tertutup. Salah satu syarat munculnya deadlock adalah circular wait. Cara mencegahnya yang paling efektif adalah menetapkan urutan penguncian yang baku: semua thread selalu mengunci kunci yang sama dengan urutan yang sama, sehingga lingkaran saling tunggu itu tidak akan pernah terbentuk.

BAB III

METODE PELAKSANAAN

3.1 Lingkungan Pengujian

Tabel 1. Lingkungan pengujian

Komponen	Keterangan
Prosesor	AMD Ryzen 5 5625U
Core fisik / core logis	6 / 12
Memori yang terlihat di WSL	2,8 GiB
Sistem operasi	Ubuntu pada WSL2 (Windows), editor VS Code
Kompiler	GCC 15.2.0
Opsi kompilasi	-O2 -fopenmp

3.2 Alat dan Bahan

Bahan praktikum adalah kerangka kode histogram.c dan deadlock.c dari modul. Kode ditulis dalam bahasa C dengan OpenMP. Berkas kode disimpan di folder src dan keluaran mentah eksperimen disimpan di folder hasil.

3.3 Langkah Percobaan

3.3.1 Pembuktian Race Condition (A1 dan A2)

Berkas src/histogram.c dikompilasi dengan perintah gcc -O2 -fopenmp src/histogram.c -o histogram. Versi race dijalankan sebanyak 5 kali dengan 8 thread (OMP_NUM_THREADS=8) dan satu kali dengan 1 thread. Kolom total dan benar pada baris race dicatat. Nilai total yang benar adalah 100.000.000.

3.3.2 Perbaikan Race Condition (C1 sampai C4)

Empat fungsi perbaikan dilengkapi pada berkas src/histogram_perbaikan.c, sedangkan kerangka src/histogram.c dibiarkan utuh sebagai bukti race condition. Kode keempat fungsi ada pada Lampiran B. Setiap fungsi diuji dahulu dengan n = 10^7 sampai menunjukkan benar=ya.

3.3.3 Pengukuran Kinerja (C5)

Kelima versi dijalankan dengan 1 thread dan 8 thread, masing-masing 3 kali, dengan n = 10^8, dan keluarannya disimpan ke hasil/histogram.txt. Nilai yang

dilaporkan adalah median dari tiga pengulangan. Versi critical dengan 8 thread membutuhkan sekitar 16,5 detik, jauh di bawah batas 2 menit pada modul, sehingga $n = 10^8$ dipakai untuk semua versi.

3.3.4 Deadlock (D1 sampai D3)

Berkas src/deadlock.c dikompilasi dengan gcc -O2 -fopenmp src/deadlock.c -o deadlock. Pada D1, fungsi transfer_ba mengunci kunci_b lebih dulu lalu kunci_a, berlawanan dengan transfer_ab. Program dijalankan 5 kali dengan perintah timeout 30, dan eksekusi yang tidak selesai dalam 30 detik dicatat sebagai macet. Pada D2, transfer_ba diubah menjadi mengunci kunci_a lebih dulu lalu kunci_b tanpa mengubah transfer_ab (berkas src/deadlock_perbaikan.c), kemudian dijalankan kembali 5 kali. Kedua versi disimpan sebagai berkas terpisah dan di-commit ke repositori.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian Race Condition

Versi race dijalankan dengan 8 thread sebanyak 5 kali pada data acak berukuran n = 100.000.000. Cuplikan keluaran terminal yang tersimpan pada hasil/histogram_a1_a2.txt ditunjukkan di bawah ini, dan rangkumannya pada Tabel 2.

```
== A1: versi race, 8 thread, 5 eksekusi ==
race       waktu=2.013 total=98582389 benar=TIDAK
race       waktu=0.646 total=97082129 benar=TIDAK
race       waktu=0.664 total=97025635 benar=TIDAK
race       waktu=0.663 total=97023812 benar=TIDAK
race       waktu=0.681 total=96856143 benar=TIDAK
== A2: versi race, 1 thread ==
race       waktu=0.037 total=100000000 benar=ya
```

Tabel 2. Versi race dengan 8 thread, n = 10^8 (total yang benar = 100.000.000)

Eksekusi	total	Benar?	Total yang hilang
1	98.582.389	tidak	1.417.611 (1,42%)
2	97.082.129	tidak	2.917.871 (2,92%)
3	97.025.635	tidak	2.974.365 (2,97%)
4	97.023.812	tidak	2.976.188 (2,98%)
5	96.856.143	tidak	3.143.857 (3,14%)
A2 (1 thread)	100.000.000	ya	0

Pada seluruh lima eksekusi dengan 8 thread, total selalu kurang dari 100.000.000 dan nilainya berbeda-beda, yaitu antara 96,86 juta dan 98,58 juta. Pada 1 thread, total tepat 100.000.000. Waktu eksekusi pertama (2,013 detik) lebih lama daripada empat eksekusi berikutnya (sekitar 0,65 detik). Hal ini kemungkinan karena pemanasan awal CPU dan cache.

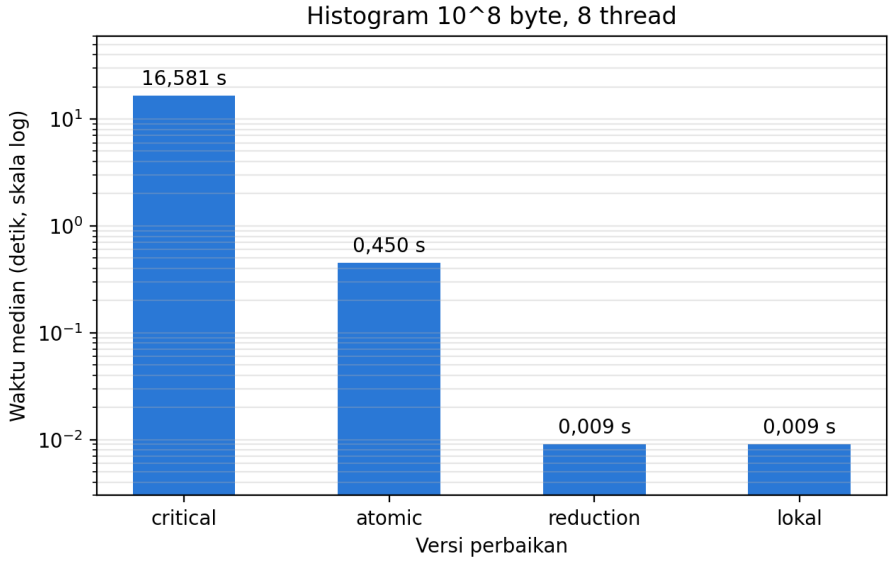
4.2 Hasil Pengukuran Kinerja Keempat Perbaikan

Keempat fungsi perbaikan menghasilkan total 100.000.000 dan benar=ya pada seluruh eksekusi, baik dengan 1 thread maupun 8 thread. Waktu eksekusi kelima versi (median 3 pengulangan, n = 10^8) disajikan pada Tabel 3. Data mentah tiga pengulangan ada pada Lampiran C.

Tabel 3. Waktu kelima versi, n = 10^8 (median 3 pengulangan, detik)

Versi	1 thread	8 thread	Speedup (1 thread / 8 thread)	Benar?
race	0,036	0,693	0,05	tidak (8 thread)
critical	0,528	16,581	0,03	ya
atomic	0,235	0,450	0,52	ya
reduction	0,036	0,009	4,0	ya
lokal	0,037	0,009	4,1	ya

Waktu reduction dan lokal pada 8 thread hanya sekitar 9 milidetik, mendekati resolusi pengukuran (1 milidetik), sehingga speedup keduanya (sekitar 4) hanya perkiraan kasar dan keduanya dianggap setara. Grafik waktu 8 thread keempat versi perbaikan ditampilkan pada Gambar 1 dengan skala logaritmik karena selisih waktunya sangat besar.



Gambar 1. Waktu 8 thread keempat versi perbaikan (skala log)

4.3 Hasil Pengujian Deadlock

Pada D1, program dijalankan 5 kali dengan batas waktu 30 detik. Seluruh eksekusi macet dan dihentikan oleh timeout. Setelah perbaikan urutan kunci (D2), seluruh eksekusi selesai dengan saldo A = 1000,0, saldo B = 1000,0, dan total saldo 2.000,0.

Keluaran D1 (sebelum perbaikan)

```
== eksekusi 1 ==
```

```
STATUS: macet atau dihentikan
== eksekusi 2 ==
STATUS: macet atau dihentikan
== eksekusi 3 ==
STATUS: macet atau dihentikan
== eksekusi 4 ==
STATUS: macet atau dihentikan
== eksekusi 5 ==
STATUS: macet atau dihentikan
```

Keluaran D2 (sesudah perbaikan)

```
== eksekusi 1 ==
saldo A = 1000.0, saldo B = 1000.0, total = 2000.0
STATUS: selesai
== eksekusi 2 ==
saldo A = 1000.0, saldo B = 1000.0, total = 2000.0
STATUS: selesai
== eksekusi 3 ==
saldo A = 1000.0, saldo B = 1000.0, total = 2000.0
STATUS: selesai
== eksekusi 4 ==
saldo A = 1000.0, saldo B = 1000.0, total = 2000.0
STATUS: selesai
== eksekusi 5 ==
saldo A = 1000.0, saldo B = 1000.0, total = 2000.0
STATUS: selesai
```

Tabel 4. Deadlock (5 eksekusi)

Versi	Selesai	Macet	Total saldo bila selesai
Sebelum perbaikan (D1)	0 dari 5	5 dari 5	-
Sesudah perbaikan (D2)	5 dari 5	0 dari 5	2.000,0

4.4 Jawaban Pertanyaan Analisis

1. Mengapa versi race menghasilkan total kurang dari n, mengapa nilainya berbeda setiap eksekusi, dan mengapa benar bila dijalankan dengan 1 thread?

Perintah h[data[i]]++ dijalankan sebagai tiga langkah, yaitu membaca h[k], menambah satu, dan menyimpan hasilnya. Bila dua thread membaca nilai h[k] yang sama sebelum salah satunya menyimpan, kedua thread menyimpan nilai lama ditambah satu sehingga satu penambahan hilang. Operasi seperti ini hanya dapat menghilangkan penambahan dan tidak pernah menambah, sehingga total selalu kurang dari n. Pada data praktikum, total berkisar antara 96,86 juta dan 98,58 juta, atau sekitar 1,4% sampai 3,1% penambahan hilang. Nilainya berbeda setiap eksekusi karena urutan penjadwalan thread tidak deterministik, sehingga jumlah tabrakan berubah-ubah. Hanya terdapat 256 bin yang diperebutkan 8 thread,

sehingga peluang tabrakan besar. Pada 1 thread hanya ada satu alur eksekusi, sehingga setiap langkah baca-tambah-simpan selesai sebelum yang berikutnya dimulai dan hasilnya selalu tepat 100.000.000.

2. Urutan keempat versi perbaikan dari yang tercepat hingga terlambat dan penjelasannya dengan konsep kontensi.

Menurut Tabel 3 pada 8 thread, urutannya adalah reduction (0,009 s) dan lokal (0,009 s) yang setara, lalu atomic (0,450 s), lalu critical (16,581 s). Reduction dan lokal tidak memiliki kontensi pada perulangan utama, karena setiap thread menambah histogram pribadinya, dan penggabungan hanya melibatkan 256 elemen per thread di akhir. Atomic tidak memakai antrean kunci karena setiap pembaruan dilakukan dengan satu instruksi perangkat keras, tetapi 8 thread masih berebut baris cache yang sama sehingga ada biaya pemindahan baris cache. Critical memaksa seluruh thread antre pada satu pintu untuk setiap dari 10^8 penambahan, sehingga biaya sinkronisasi jauh lebih besar daripada pekerjaan yang dilindungi, yaitu satu penambahan. Rata-rata, satu penambahan pada critical dengan 8 thread membutuhkan sekitar 166 nanodetik, sedangkan dengan 1 thread hanya sekitar 5 nanodetik.

3. Kemiripan reduction dan lokal serta satu kelebihan menulis versi lokal sendiri.

Keduanya memberi setiap thread salinan histogram sendiri, mengisinya tanpa rebutan, lalu menggabungkan semua salinan sekali di akhir. Perbedaannya hanya pada siapa yang mengatur salinan dan penggabungannya: pada reduction oleh kompiler, pada lokal oleh programmer. Kelebihan menulis versi lokal sendiri adalah programmer mengendalikan cara penggabungan, misalnya memberi padding agar histogram setiap thread tidak berbagi baris cache. Cara ini juga tidak bergantung pada dukungan reduction pada array (OpenMP 4.5), sehingga dapat dipakai pada kompiler yang lebih lama.

4. Mengapa versi critical bisa lebih lambat dengan 8 thread daripada dengan 1 thread?

Pada 1 thread, critical hanya membayar biaya masuk dan keluar wilayah kritis (0,528 s). Pada 8 thread, thread harus menunggu giliran dan variabel kunci terus

berpindah antar core, sehingga waktunya menjadi 16,581 s, sekitar 31 kali lebih lambat. Pekerjaan yang dilindungi hanya satu penambahan, sehingga hampir seluruh waktu habis untuk sinkronisasi dan tidak ada keuntungan dari paralelisme. Dengan kata lain, 8 thread malah bekerja lebih lambat daripada program serial.

5. Mengapa deadlock tidak selalu terjadi pada setiap eksekusi, dan syarat deadlock mana yang dipatahkan oleh perbaikan?

Deadlock hanya terjadi bila pada saat yang bersamaan thread pertama sudah memegang kunci A dan menunggu kunci B, sedangkan thread kedua sudah memegang kunci B dan menunggu kunci A. Kejadian ini bergantung pada penjadwalan thread. Pada percobaan ini, setiap thread melakukan satu juta iterasi sehingga peluang bersinggungan sangat besar, dan hasilnya 5 dari 5 eksekusi macet. Bila iterasinya sedikit, satu thread dapat selesai lebih dulu sebelum thread lain mulai dan program lolos. Perbaikan mematahkan syarat circular wait. Dengan urutan penguncian yang tetap (selalu A lebih dulu, lalu B), tidak ada thread yang memegang B sambil menunggu A, sehingga rantai tunggu melingkar tidak dapat terbentuk.

BAB V

PENUTUP

5.1 Kesimpulan

1. **Race condition terbukti:** versi race dengan 8 thread menghasilkan total 96,86 juta sampai 98,58 juta dari 100.000.000 dan berbeda setiap eksekusi akibat lost update pada langkah baca-tambah-simpan. Dengan 1 thread, hasilnya tepat 100.000.000.

1. **Keempat perbaikan benar:** critical, atomic, reduction, dan akumulasi lokal menghasilkan total 100.000.000 pada 1 thread dan 8 thread.

2. **Kinerja ditentukan kontensi:** pada 8 thread, reduction dan lokal tercepat (0,009 s), disusul atomic (0,450 s), dan critical paling lambat (16,581 s), bahkan sekitar 31 kali lebih lambat daripada 1 thread.

3. **Deadlock dicegah dengan urutan kunci tetap:** sebelum perbaikan 5 dari 5 eksekusi macet. Setelah transfer_ba mengunci A lebih dulu lalu B, 5 dari 5 eksekusi selesai dengan total saldo 2.000,0 karena syarat circular wait dipatahkan.

5.2 Saran

1. Hindari critical di dalam perulangan dengan iterasi sangat banyak. Gunakan atomic bila hanya satu pembaruan sederhana yang dilindungi.
2. Untuk akumulasi data berskala besar, utamakan reduction atau array lokal per thread agar tidak ada rebutan memori bersama.
3. Pada program yang memakai lebih dari satu kunci, tetapkan urutan penguncian yang sama di seluruh fungsi untuk mencegah deadlock.
4. Untuk pengukuran yang lebih teliti pada waktu di bawah 10 milidetik, gunakan n yang lebih besar atau jumlah pengulangan yang lebih banyak.

LAMPIRAN

Lampiran A. Struktur Berkas

```
p2-sinkronisasi/
  src/
    histogram.c               kerangka (versi serial dan race)
    histogram_a1_kerangka.c   salinan kerangka untuk A1 dan A2
    histogram_perbaikan.c     C1-C4: critical, atomic, reduction, lokal
    deadlock.c                D1: transfer_ba mengunci B lalu A
    deadlock_perbaikan.c      D2: transfer_ba mengunci A lalu B
  hasil/
    histogram_a1_a2.txt       bukti race (5 run 8 thread, 1 run 1
thread)
    histogram.txt             C5: kelima versi, 1 dan 8 thread, 3
ulangan
    deadlock_d1.txt           5 eksekusi sebelum perbaikan
    deadlock_d2.txt           5 eksekusi sesudah perbaikan
  README.md
```

Lampiran B. Kode Perbaikan

```
/* C1: critical */
#pragma omp parallel for
for (long i = 0; i < n; i++) {
    #pragma omp critical
    h[data[i]]++;
}

/* C2: atomic */
#pragma omp parallel for
for (long i = 0; i < n; i++) {
    #pragma omp atomic
    h[data[i]]++;
}

/* C3: reduction pada array */
#pragma omp parallel for reduction(+:h[:BIN])
for (long i = 0; i < n; i++) h[data[i]]++;

/* C4: histogram lokal per thread */
#pragma omp parallel
{
    long lokal[BIN];
    memset(lokal, 0, sizeof(lokal));
    #pragma omp for
    for (long i = 0; i < n; i++) lokal[data[i]]++;
    for (int k = 0; k < BIN; k++) {
        #pragma omp atomic
        h[k] += lokal[k];
    }
}
```

Fungsi transfer_ba pada D1 dan D2 (transfer_ab tidak diubah):

```
/* D1: macet (B lebih dulu, lalu A) */
void transfer_ba(double x) {
    omp_set_lock(&kunci_b);
    omp_set_lock(&kunci_a);
```

```
    saldo_b -= x;  saldo_a += x;
    omp_unset_lock(&kunci_a);
    omp_unset_lock(&kunci_b);
}

/* D2: aman (A lebih dulu, lalu B, sama dengan transfer_ab) */
void transfer_ba(double x) {
    omp_set_lock(&kunci_a);
    omp_set_lock(&kunci_b);
    saldo_b -= x;  saldo_a += x;
    omp_unset_lock(&kunci_b);
    omp_unset_lock(&kunci_a);
}
```

Lampiran C. Data Mentah Tiga Pengulangan (n = 10^8, detik)

Versi	Thread	Ulangan 1	Ulangan 2	Ulangan 3	Median
race	1	0,043	0,036	0,035	0,036
critical	1	0,528	0,546	0,523	0,528
atomic	1	0,236	0,234	0,235	0,235
reduction	1	0,040	0,036	0,036	0,036
lokal	1	0,049	0,037	0,036	0,037
race	8	0,675	0,700	0,693	0,693
critical	8	16,581	16,617	16,467	16,581
atomic	8	0,443	0,472	0,450	0,450
reduction	8	0,008	0,011	0,009	0,009
lokal	8	0,011	0,009	0,009	0,009

Lampiran D. Link Repository Github

Link: https://github.com/Chioaji/Pemrosesan_Pararel_P2_Sinkronasi